

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

1. Introducción

Los objetivos de esta práctica son los siguientes:

1. Introducción a las herramientas que facilitan la automatización de pruebas con interfaz gráfica de usuario. En particular, se trabajará con las herramientas Espresso, UI Automator, Exerciser Monkey y Selenium.
2. Introducción a las herramientas que facilitan la automatización de pruebas de aceptación de usuario. En particular, se trabajará con la herramienta Cucumber.
3. Utilización de estas herramientas para automatizar algunas pruebas globales (pruebas de sistema o de aceptación) de la aplicación que se utiliza como objeto de pruebas en el trabajo de la asignatura.

2. Actividades a realizar en la práctica

2.1. Trabajo con la herramienta Espresso

2.1.1. Creación de un proyecto de demostración

Vais a ver cómo se pueden crear *scripts* de pruebas con Espresso adaptando el tutorial sobre Espresso disponible en el portal Vogella [8]. La aplicación que probaréis consta de dos actividades. La actividad principal consta de un cuadro de texto y dos botones. El primer botón pone el texto fijo «Lalala» en un cuadro de texto. El segundo de ellos debe lanzar la segunda actividad, que se limita a mostrar el texto del cuadro de texto.

Aunque el proyecto de demostración está disponible en GitHub (<https://github.com/unizar-vv-organization/unizar-vv-practica6-android>), si tuvieseis que crearlo desde



Práctica 6: Automatización de pruebas sobre la interfaz de usuario

cero seguiríais los siguientes pasos:

- Cread un proyecto Android utilizando la plantilla *Empty Views Activity* como base del proyecto con una actividad principal (*MainActivity*).
- Configurad las dependencias de pruebas de Android (*androidTestImplementation*) en el módulo correspondiente a la aplicación.¹ En el fichero «*build.gradle*» de la aplicación (utilizando el estilo *Groovy DSL*) hay que especificar las siguientes dependencias:

```
dependencies {
    ...
    // Assertions
    androidTestImplementation 'androidx.test.ext:junit:1.2.1'
    // UI testing with Espresso
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.6.1'
}
```

Para utilizar clases de JUnit, *AndroidJUnitRunner* tiene que estar especificado como el ejecutor por defecto en el proyecto, a través de la siguiente declaración en el fichero «*build.gradle*» del módulo correspondiente a la aplicación:

```
android {
    ...
    defaultConfig {
        ...
        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
    }
}
```

- Modificad el layout de la actividad principal («*activity_main.xml*») para que contenga lo siguiente:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <EditText
        android:id="@+id/inputField"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
    <Button
        android:id="@+id/changeText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

¹Se puede consultar más información sobre la configuración de AndroidX Test Library en <https://developer.android.com/training/testing/instrumented-tests/androidx-test-libraries/test-setup>

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

```

        android:text="New Button" android:onClick="onClick"/>
    <Button
        android:id="@+id/switchActivity"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Change Text" android:onClick="onClick"/>
</LinearLayout>

```

- Cread una segunda actividad *SecondActivity* de tipo *Empty Views Activity* y modificad su layout («*activity_second.xml*») de la siguiente forma:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical" android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textAppearance="?android:attr/textAppearanceLarge"
        android:text="Large Text"
        android:id="@+id/resultView" />
</LinearLayout>

```

- Modificad el código de la actividad *SecondActivity* con el que aparece a continuación:

```

import ...;

public class SecondActivity extends AppCompatActivity{
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
        TextView viewById = findViewById(R.id.resultView);
        Bundle inputData = getIntent().getExtras();
        String input = inputData.getString("input");
        viewById.setText(input);
    }
}

```

- Modificad también el código de la actividad principal (*MainActivity*) de la siguiente forma:

```

import ...;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
    }
}

```

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

```

        setContentView(R.layout.activity_main);
    }
    public void onClick(View view) {
        if (view.getId() == R.id.changeText) {
            EditText editText = findViewById(R.id.inputField);
            editText.setText("Lalala");
        } else if (view.getId() == R.id.switchActivity) {
            Intent intent = new Intent(this, SecondActivity.class);
            startActivity(intent);
        }
    }
}

```

- Cread ahora un *script* de pruebas con Espresso para probar el GUI de la aplicación que acabas de construir. Este *script* se incluirá en una una clase denominada MainActivityEspressoTest (dentro de la carpeta «src/androidTest/java» junto al resto de pruebas instrumentadas) con el siguiente contenido:

```

import ...;

public class MainActivityEspressoTest {
    @Rule
    public ActivityScenarioRule<MainActivity> mActivityRule
        = new ActivityScenarioRule<>(MainActivity.class);
    @Test
    public void ensureTextChangesWork() {
        // Type text and then press the button.
        onView(withId(R.id.inputField))
            .perform(typeText("HELLO"), closeSoftKeyboard());
        onView(withText("Change Text")).perform(click());

        // Check that the text was changed.
        onView(withId(R.id.inputField)).check(matches(withText("Lalala")));
    }
    @Test
    public void ensureLaunchSecondActivityWithInputTextWork() {
        // Type text and then press the button.
        onView(withId(R.id.inputField)).perform(typeText("NewText"),
            closeSoftKeyboard());
        onView(withText("Switch Activity")).perform(click());

        // This view is in a different Activity, no need to tell Espresso.
        onView(withId(R.id.resultView)).check(matches(withText("NewText")));
    }
}

```

En el código anterior puedes observar que se instancia un objeto de la clase ActivityScenarioRule. ActivityScenarioRule minimiza la cantidad de código que hay que escribir ya que le indica al *framework* que debe lanzar la actividad objeto

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

de las pruebas con anterioridad a los métodos anotados con `@Before` y `@Test`. El *framework* también se encarga de finalizar la actividad después de que se hayan finalizado los tests y todos los métodos anotados con `@After`.

- Antes de ejecutar el test, chequead que en el **fichero «gradle.properties»** del proyecto se ha activado la característica `android.injected.androidTest.leaveApksInstalledAfterRun` para que el complemento de Gradle para Android no desinstale la aplicación después de realizar pruebas, añadiendo la siguiente declaración:

```
android.injected.androidTest.leaveApksInstalledAfterRun=true
```

- Ejecutad ahora el test que acabáis de crear. Para ejecutar pruebas instrumentadas en Android, en la ventana del proyecto, hay que buscar la clase o método con la prueba instrumentada y ejecutarlos con el botón derecho del ratón. Para ejecutar todos los tests de un directorio, se selecciona el directorio con el botón derecho del ratón y se elige el botón de ejecutar.
- Como comprobaréis, los tests no pasan: hay varios errores en las actividades que acabáis de construir que se deberían corregir para superar el test, pero no hay errores en la clase `MainActivityEspressoTest`.

Ahora probaréis a crear un test adicional para comprobar la interacción con otras actividades. Con esta opción, denominada *mock intents*, se puede verificar que una actividad lanza correctamente *intents* y reacciona correctamente si recibe los resultados de un `Intent`. Para realizar este test seguiréis los siguientes pasos:

- Aseguraos de incluir la siguiente dependencia en el fichero «`build.gradle`» de la aplicación:

```
dependencies {
    ...
    // Intent mocking support
    androidTestImplementation "androidx.test.espresso:espresso-intents:3.6.1"
}
```

- Modificad la clase `MainActivity` para incluir un botón adicional con el texto «*Send email*» que se encargue de enviar un correo al pulsar el botón con el siguiente código:

```
Intent emailIntent = new Intent(android.content.Intent.ACTION_SEND);
emailIntent.putExtra(android.content.Intent.EXTRA_SUBJECT, "Subject");
emailIntent.setType("plain/text");
emailIntent.putExtra(android.content.Intent.EXTRA_TEXT, "Body Text");
startActivity(Intent.createChooser(emailIntent, "Send mail..."));
```

- Cread ahora una clase denominada `TestIntent` para verificar si se lanza la actividad de envío de correo correctamente con el siguiente código:

```
import ...;

public class TestIntent {
    @Rule
    public ActivityScenarioRule<MainActivity> mActivityRule =
        new ActivityScenarioRule<>(MainActivity.class);
    @Before
    public void setUp() {
        Intents.init();
    }
    @After
    public void tearDown() {
        Intents.release();
    }
    @Test
    public void triggerIntentTest() {
        onView(withId(R.id.sendEmail)).perform(click());
        intended(hasAction(equalTo(Intent.ACTION_CHOOSER)));
        // se podría utilizar toPackage si conociésemos a priori el paquete que
        // contiene a la actividad que se lance:
        // intended(toPackage("paqueteActividadLanzada"));
    }
}
```

Comprobaréis que ahora hemos utilizado la clase `Intents` para permitir que Espresso pueda lanzar *intents* (se configura en el método de inicialización, etiqueta *@before*, y el método de finalización, etiqueta *@after*). También se puede observar en el test como el método `intended` permite verificar si se ha lanzado o no el `Intent` correcto.

- Ejecutad ahora el test y verificar que pasa correctamente.

Puede ocurrir que alguno tengáis algún problema para ejecutar los test si trabajáis con el emulador o el dispositivo móvil. Estos problemas son debidos a los efectos de las animaciones de vuestros sistemas. Para que los test se ejecuten correctamente, debéis desactivar las animaciones a través de las opciones de desarrollo en los ajustes del emulador o del dispositivo móvil.²

2.1.2. Creación de un proyecto para probar el GUI de la aplicación del trabajo de la asignatura

Probad a utilizar la herramienta Espresso para automatizar alguna de las pruebas de sistema que hayáis planteado para la aplicación del trabajo de la asignatura. Al menos, se debería utilizar Espresso para automatizar la prueba de sistema diseñada con la técnica

²<https://developer.android.com/training/testing/espresso/setup>

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

de prueba de caminos sobre el mapa de navegación.

Como punto de partida se facilita en el fichero «`EspressoTest.java`» un ejemplo inicial de un test sobre la aplicación de gestión de notas (proporcionada como ejemplo en la práctica 2 de la asignatura de Ingeniería del Software) que crea y actualiza un conjunto de notas a través de la interfaz de la aplicación. Además, comprueba de dos formas distintas que dichas notas se han creado (verificando que aparecen en el listado de notas de la actividad principal, por una parte, y comprobando que aparecen también en la base de datos). Posteriormente, las notas creadas se borran, también a partir de la interfaz. Este test se ha probado que funciona correctamente con un Pixel 3a API 35 (`compileSdk=35`, `targetSdk=34`).

Seguramente la estructura de la interfaz de vuestra aplicación tendrá diferencias respecto a la que espera el test de ejemplo que os suministramos. En ese caso, corregid lo que sea necesario para que el test pase. También debéis comprobar si el fichero «`build.gradle`» de vuestra aplicación contiene ya todas las dependencias necesarias para ejecutar pruebas con Espresso.

Se recomienda consultar la documentación para desarrolladores de Espresso.³ En particular, puede ser de interés echar un vistazo a los ejemplos de utilización del método `onData`, en lugar de `onView`, para establecer emparejamientos con los datos que se muestran dentro de una lista.⁴

2.2. Trabajo con la herramienta UI Automator

2.2.1. Creación de un proyecto de demostración

En base al tutorial disponible en el portal Vogella [7], vamos a ver cómo crear un *script* de pruebas utilizando la herramienta UI Automator.

Aunque el *script* de pruebas con UI Automator está disponible en GitHub(<https://github.com/unizar-vv-organization/unizar-vv-practica6-android>), si tuvieseis que incorporarlo al proyecto de demostración descrito en la sección 2.1.1, seguiríais los siguientes pasos:

- Aseguraos de que existe la siguiente dependencia en el fichero «`build.gradle`» de la aplicación:

```
dependencies {  
    ...  
    // UI testing with UI Automator  
    androidTestImplementation "androidx.test:uiautomator:2.3.0"
```

³<https://developer.android.com/training/testing/espresso>

⁴<https://developer.android.com/training/testing/espresso/basics#checking-data-adapter-views>,
<https://developer.android.com/training/testing/espresso/additional-resources#samples>

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

```
}
```

- Cread ahora una clase denominada `MyUiAutomatorTest` cuyo objetivo es comprobar que el botón “*Change Text*” funciona correctamente. Es un test equivalente al test `ensureTextChangesWork` planteado en la clase `MainActivityEspressoTest`, pero ahora planteado como un test de caja negra con UI Automator:

```
import ...;

@RunWith(AndroidJUnit4.class)
public class MyUiAutomatorTest {
    private UiDevice mDevice;
    private static final int TIMEOUT = 5000;
    final private static String APP_PACKAGE = "es.unizar.eina.demonstrator";

    @Before
    public void startMainActivityFromHomeScreen() {
        // Se inicializa la instancia de UiDevice
        mDevice = UiDevice.getInstance(InstrumentationRegistry.getInstrumentation());
        // Se pulsa el botón para ir a la pantalla inicial
        mDevice.pressHome();
        // Esperar a que el lanzador de aplicaciones esté disponible
        final String launcherPackage = mDevice.getLauncherPackageName();
        // launcherPackageName toma el valor "com.android.settings"
        assertEquals(launcherPackage, notNullValue());
        mDevice.wait(Until.hasObject(By.pkg(launcherPackage).depth(0)), TIMEOUT);
        // Abrir la aplicación
        Context context = ApplicationProvider.getApplicationContext();
        final Intent intent = context.getPackageManager()
            .getLaunchIntentForPackage(APP_PACKAGE);
        // Configurar que se elimine cualquier instancia previa
        intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TASK);
        context.startActivity(intent);
        // Esperar hasta que el dispositivo esté disponible
        mDevice.wait(Until.hasObject(By.pkg(APP_PACKAGE).depth(0)), TIMEOUT);
    }

    /**
     * Alternativa de método Before para abrir la aplicación haciendo clic sobre su nombre
     * en la pantalla donde se muestran todas las aplicaciones disponibles
     */
    public void startMainActivityFromHomeScreenClickingOnAppName() throws UiObjectNotFoundException {
        // Se inicializa la instancia de UiDevice
        mDevice = UiDevice.getInstance(InstrumentationRegistry.getInstrumentation());
        // Se pulsa el botón para ir a la pantalla inicial
        mDevice.pressHome();
        // Nos deslizamos por la pestaña en sentido vertical (por defecto) u horizontal
        UiScrollable appViews = new UiScrollable(new UiSelector()
            .scrollable(true));
    }
}
```


Práctica 6: Automatización de pruebas sobre la interfaz de usuario

```

        appViews.scrollForward();
        // Abrir la aplicación
        mDevice.wait(Until.hasObject(By.desc(APP_NAME)), TIMEOUT);
        UiObject2 app = mDevice.findObject(By.desc(APP_NAME));
        assertThat(app, notNullValue());
        app.clickAndWait(Until.newWindow(), TIMEOUT);
    }

    @Test
    public void ensureTextChangesWork() {
        // Localizar la caja de texto y teclear un texto
        UiObject2 editText = mDevice.findObject(By.clazz("android.widget.EditText"));
        editText.setText("Hello");
        // Pulsar el botón
        UiObject2 changeTextButton = mDevice.findObject(By.text("Change Text"));
        changeTextButton.click();
        // Verificar que ha cambiado el valor localizando caja de texto por su contenido
        String expectedText = "Lalala";
        mDevice.wait(Until.hasObject(By.text(expectedText)), TIMEOUT);
        UiObject2 lalala = mDevice.findObject(By.text(expectedText));
        assertThat(lalala, notNullValue());
    }
}

```

Podéis comprobar en este ejemplo que se utilizan las aserciones de JUnit, en particular, las de la biblioteca Hamcrest (`assertThat`).

- Ejecutad el test y verificad que pasa correctamente.
- En caso de que haya habido algún problema, utiliza la herramienta UI Automator Viewer para analizar las pantallas que se muestran en el dispositivo y inspeccionar los atributos (ej: `android:contentDescription`) que caracterizan a cada componente gráfico. Para abrir esta herramienta, hay que ejecutar, desde un terminal o símbolo del sistema, la siguiente orden:

```
uiautomatorviewer
```

La herramienta `uiautomatorviewer` está dentro del directorio donde esté instalado el SDK de Android, en el subdirectorio «`Sdk/tools/bin`». Si no tenéis instalada la versión 8 de Java, es probable que tengáis algún problema en la ejecución de la herramienta. Podéis descargarla e instalarla en vuestros equipos.⁵ Por ejemplo, en Ubuntu se puede instalar y configurar su uso entre las máquinas virtuales instaladas con los siguientes comandos:

```

sudo apt-get install temurin-8-java
sudo update-alternatives --config java

```

⁵<https://adoptium.net/temurin/releases/?version=8>

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

Una vez abierta la herramienta, se puede capturar la pantalla del dispositivo con el botón *Device Screenshot* tal como se muestra en la figura 1.

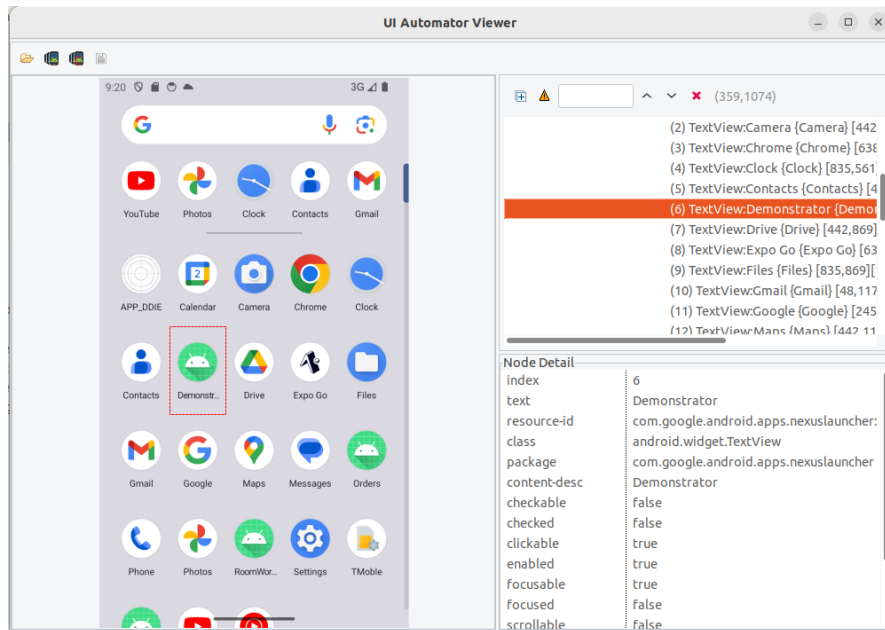


Figura 1: Herramienta UI Automator Viewer

En el caso de que no sea posible capturar directamente desde `uiautomatorviewer` la pantalla del dispositivo, lo que debemos hacer es capturar la imagen de la pantalla en formato PNG y el fichero con el *layout* en formato XML a través de los siguientes comandos en línea con la herramienta *adb* (*android debug bridge*) ubicada en el subdirectorio «`Sdk/platform-tools/`»:

```
adb shell uiautomator dump /sdcard/test.uix
adb pull /sdcard/test.uix
adb shell screencap -p /sdcard/test.png
adb pull /sdcard/test.png
```

Posteriormente, utilizaremos `uiautomatorviewer` para visualizar los ficheros generados tal como se muestra en la figura 2 pulsando la opción de menú **Open**. También será seguramente necesario modificar el procedimiento para explorar las aplicaciones que se utilizaba anteriormente.

2.2.2. Creación de un proyecto para probar el GUI de la aplicación del trabajo de la asignatura

Probad a utilizar la herramienta UI Automator para automatizar alguna de las pruebas de sistema que hayáis planteado para la aplicación del trabajo de la asignatura.

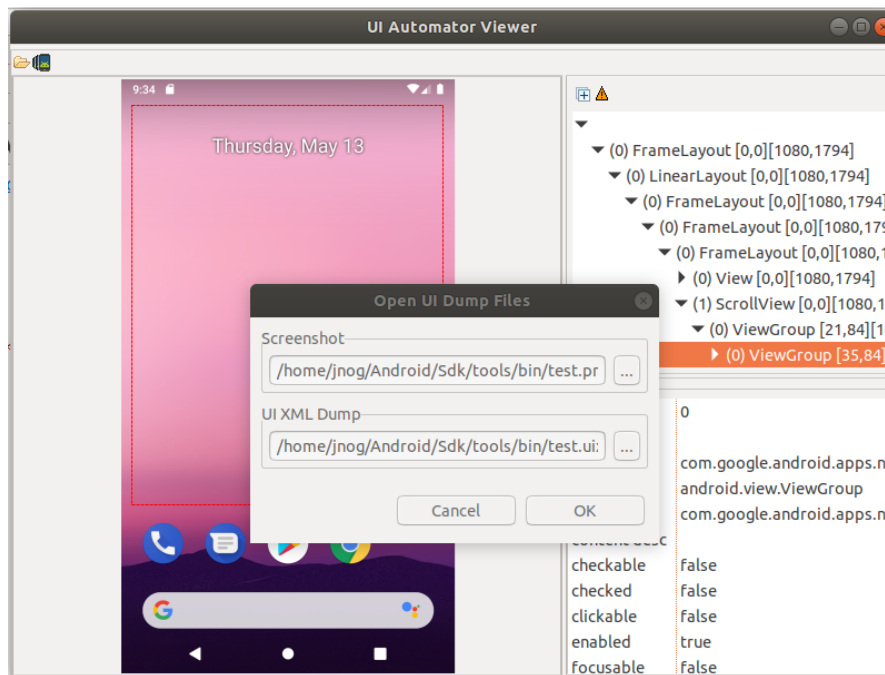


Figura 2: Visualización de pantalla y *layout* en UI Automator Viewer con ficheros generados previamente.

Recordad que podéis utilizar la herramienta UI Automator Viewer para analizar las pantallas que queréis probar posteriormente y encontrar los atributos que identifican a cada elemento gráfico. Además, deberéis verificar que en el *layout* de las distintas actividades los elementos gráficos están anotados con atributos *android:contentDescription* en el caso de botones, o atributos *android:hint* en el caso de cajas de texto. Se recomienda consultar la documentación para desarrolladores de UI Automator⁶ para obtener más información sobre las distintas posibilidades de la herramienta.

2.3. Trabajo con la herramienta Cucumber

2.3.1. Creación de un proyecto de demostración con IntelliJ

Vais a crear desde el IDE IntelliJ un proyecto de demostración de utilización del *framework* de pruebas Cucumber adaptando un tutorial del portal de Think Code [6].

Aunque el script de pruebas está disponible en GitHub (<https://github.com/unizar-vv-organization/unizar-vv-practica6-cucumber>), si tuvieséis que crearlo desde cero seguiríais los siguientes pasos:

⁶<https://developer.android.com/training/testing/other-components/ui-automator>

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

1. Cread desde IntelliJ un proyecto Java de tipo Gradle (DSL basado en Groovy). En `Advanced Settings` debes incluir también la información de *GroupId* (ej: se.waymark.educational), *ArtifactId* (ej: RentACar) y nombre del proyecto (ej: RentACar).
2. Aseguraos de que el plugin *Cucumber for Java* está activado a través de la opción `File > Settings > Plugins`. Si no aparece el plugin, búscalo e instálalo.
3. Modificad el fichero «`build.gradle`» del proyecto para incluir la siguiente configuración de acuerdo a las indicaciones de uso de Cucumber con proyectos Java configurados con Gradle:⁷

```

plugins {
    id 'java'
}

group 'se.waymark.educational'
version '1.0-SNAPSHOT'

repositories {
    mavenCentral()
}

dependencies {
    testImplementation 'junit:junit:4.13.2'
    testImplementation 'io.cucumber:cucumber-java:7.22.0'
}

configurations {
    cucumberRuntime {
        extendsFrom testImplementation
    }
}

task cucumber() {
    dependsOn assemble, testClasses
    doLast {
        javaexec {
            main = "io.cucumber.core.cli.Main"
            classpath = configurations.cucumberRuntime + sourceSets.main.output +
                ⇨ sourceSets.test.output
            args = ['--plugin', 'pretty', '--glue', 'se.waymark.rentit.steps', '
                ⇨ src/test/resources/features']
        }
    }
}

```

⁷<https://cucumber.io/docs/tools/java/#build-tools>

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

Importa las dependencias a través de la ventana del plugin de Gradle (**View** **Tool Windows** **Gradle**) con la opción **Sync All Gradle Projects**.

4. Cread el fichero «Rent.feature» dentro de la carpeta «src/test/resources/features» con el menú contextual **New** **File** con el siguiente contenido describiendo la funcionalidad de alquiler de coches y un escenario asociado utilizando el lenguaje Gherkin:⁸

```
Feature: Rental cars should be possible to rent to gain revenue to the rental
    ↪ company.
```

```
As an owner of a car rental company
I want to make cars available for renting
So I can make money
```

```
Scenario: Find and rent a car
    Given there are 18 cars available for rental
    When I rent one
    Then there will only be 17 cars available for rental
```

5. Ejecuta el fichero «Rent.feature» con el menú contextual **Run**. Verás que como salida de la ejecución se van creando métodos con anotaciones **@Given**, **@When** y **@Then**. Crea la clase Java `se.waymark.rentit.steps.RentStepsDefinition` en la carpeta «src/test/java» e integra estos métodos dentro de la clase.
6. Incorporad el código Java que se facilita en GitHub para las clases e interfaces `RentACarSupport`, `CarDAO`, `InMemoryCarDAO` y `Car` a la carpeta «src/main/java».
7. Probad a ejecutar los tests con la opción **Run** del menú contextual sobre el «Rent.feature». Si falla puede ser porque no hemos especificado en el entorno de desarrollo el paquete donde se encuentra la implementación de los pasos. Utilizad la opción **Run** **Edit Configurations** para poner el paquete `se.waymark.rentit.steps` dentro de la caja de texto *Glue*.

Otra posibilidad para ejecutar los tests es ejecutar la tarea `cucumber` dentro del fichero «build.gradle»: o bien desde IntelliJ utilizando el icono de ejecución de la tarea, o bien con el comando `gradle cucumber` en una terminal y sobre el directorio donde se encuentra el fichero «build.gradle».

8. La ejecución de los test seguirá dando fallos. Implementad ahora realmente los pasos del test con el siguiente código y comprobad que los test funcionan:

```
package se.waymark.rentit.steps;
```

⁸Lenguaje mínimamente controlado que utiliza texto plano y un conjunto limitado de palabras reservadas para especificar el comportamiento esperado del software, ver <https://cucumber.io/docs/gherkin/reference>.

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

```

import io.cucumber.java.en.Given;
import io.cucumber.java.en.Then;
import io.cucumber.java.en.When;
import se.waymark.rentit.RentACarSupport;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.core.Is.is;

public class RentStepsDefinition {
    private RentACarSupport rentACarSupport = new RentACarSupport();
    @Given("there are {int} cars available for rental")
    public void there_are_cars_available_for_rental(Integer availableCars) {
        rentACarSupport.createCars(availableCars);
    }
    @When("I rent one")
    public void i_rent_one() {
        rentACarSupport.rentACar();
    }
    @Then("there will only be {int} cars available for rental")
    public void there_will_only_be_cars_available_for_rental(Integer expectedAvailableCars) {
        int actualAvailableCars = rentACarSupport.getAvailableNumberOfCars();
        assertThat(actualAvailableCars, is(expectedAvailableCars));
    }
}

```

9. Probad ahora a añadir una nueva funcionalidad para devolver coches en el fichero «Return.feature» con el siguiente texto y ejecutad el fichero para identificar los pasos que faltan por implementar:

Feature: Return of rented cars should be possible to ensure sustainability of the rental company.

As an owner of a car rental company
 I want to make cars available **for** renting
 and get them returned
 So I can make money

Scenario: Find and rent a car
 Given there are 18 cars available **for** rental
 When I rent one
 And I **return** one
 Then there will only be 18 cars available **for** rental

10. Implementad los pasos adicionales y modificad el código fuente para ampliar la funcionalidad de la aplicación.
11. Una vez implementados los pasos nuevos, probad ahora a reconvertir el *Scenario* en un *Scenario Outline* con el menú contextual. Verás que esta reconversión nos permite expresar casos de prueba parametrizados. Los nombres de los parámetros se corresponden con los identificadores de los parámetros en la definición de los

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

pasos. Podemos añadir tantas filas como deseemos debajo de *Examples* para ejecutar el mismo caso de pruebas con otros valores. Si añadís varias filas, seguramente detectaréis un problema en la implementación de la clase `InMemoryCarDAO`. Tratad de resolverlo.

```
Scenario Outline: Find and rent a car
  Given there are <availableCars> cars available for rental
  When I rent one
  And I return one
  Then there will only be <expectedAvailableCars> cars available for rental
Examples:
  | availableCars | expectedAvailableCars |
  | 18            | 18                     |
```

Los puntos 9, 10 y 11 no están resueltos en el repositorio inicial. Si encontráis dificultades en su resolución, podéis clonar el repositorio disponible en <https://github.com/unizar-vv-organization/unizar-vv-practica6-cucumber-solucion>.

2.3.2. Integración de Cucumber y Espresso en Android Studio

Vais a ver cómo extender el proyecto de demostración descrito en la sección 2.1.1 para añadir una prueba de aceptación automatizada utilizando de forma combinada Cucumber y Espresso.

Aunque esta prueba está integrada directamente en el repositorio <https://github.com/unizar-vv-organization/unizar-vv-practica6-android>, si tuvieseis que incorporarla al proyecto de demostración descrito en la sección 2.1.1, seguiríais los siguientes pasos:

1. En el fichero «`build.gradle`» de la aplicación, tenéis que añadir las siguientes dependencias:

```
dependencies {
    ...
    androidTestImplementation 'io.cucumber:cucumber-android:7.14.0'
    androidTestImplementation 'io.cucumber:cucumber-picocontainer:7.17.0'
}
```

2. También hay que indicar en «`build.gradle`» que el ejecutor de las pruebas instrumentadas es `CucumberAndroidJUnitRunner` y comentar el uso de `AndroidJUnitRunner`:

```
android {
    ...
    defaultConfig {
        ...
        testInstrumentationRunner "io.cucumber.android.runner.
            ↪ CucumberAndroidJUnitRunner"
```

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

```
    }
}
```

3. Crearéis la definición de una *feature* con el flujo principal (o escenario) del caso de uso. Por ejemplo, en el caso de probar la aplicación de gestión de notas, el *feature* para la creación de una nota en el fichero «app/src/androidTest/assets/features/demonstrator.feature» tendría el siguiente contenido:

Feature: La aplicación es capaz de cambiar el texto de forma automática
 Se actualiza la caja de texto de MainActivity con el texto 'Lalala' al pulsar
 ↪ el botón 'Change text'

Scenario Outline: Teclear texto nuevo, pulsar botón 'Change text'
 Given Abro la aplicación Demonstrator
 When Tecleo "<textoTecleado>" en la caja de texto
 And Pulso el botón "<etiqueta>"
 Then Debería ver "<textoCambiado>"

Examples:

textoTecleado	etiqueta	textoCambiado
Hola	Change Text	Lalala
Adios	Change Text	Lalala

4. Crearéis la clase `RunCukesTest` dentro del subpaquete `test` del paquete donde está contenida la actividad principal de vuestra aplicación (en el ejemplo, esta clase estaría ubicada en la carpeta «app/src/androidTest/es/unizar/eina/demonstrator/test»), con una anotación indicando donde se encuentran las definiciones de features (carpeta «features» dentro de «app/src/androidTest/assets»):

```
package es.unizar.eina.demonstrator.test;
import io.cucumber.junit.CucumberOptions;
@CucumberOptions(
    features = {"features"}
)
public class RunCukesTest {
}
```

5. Crearéis una clase con la definición de los pasos del feature que se ha definido previamente dentro del mismo paquete y carpeta que la clase anterior. Por ejemplo, en el caso de la aplicación de gestión de notas la implementación de los pasos sería la siguiente:

```
import ...;

public class RunStepsDefinition {
    ActivityTestRule rule = new ActivityTestRule<>(MainActivity.class);
    @Before
```


Práctica 6: Automatización de pruebas sobre la interfaz de usuario

```

    public void launchActivity() throws Exception {
        rule.launchActivity(null);
    }
    @After
    public void finishActivity() throws Exception {
        rule.getActivity().finish();
    }
    @Given("Abro la aplicación Demonstrator")
    public void abro_la_aplicacion_Demonstrator() {
        assertNotNull(rule.getActivity());
    }
    @When("Tecleo {string} en la caja de texto")
    public void tecleo_textoTecleado_en_la_caja_de_texto(final String
        ↪ textoTecleado) {
        onView(withId(R.id.inputField)).perform(replaceText(textoTecleado),
        ↪ closeSoftKeyboard());
    }
    @When("Pulso el botón {string}")
    public void pulso_el_boton_etiqueta(final String etiqueta) {
        onView(withText(etiqueta)).perform(click());
    }
    @Then("Debería ver {string}")
    public void deberia_ver_textoCambiado(final String textoCambiado) {
        // Aserción: comprobación de que el texto esperado aparece
        onView(withText(textoCambiado)).check(matches(isDisplayed()));
    }
}

```

6. Probad a ejecutar la *feature* con el menú contextual sobre la carpeta *features* en Android Studio.

2.3.3. Creación de un proyecto para realizar pruebas de aceptación sobre el GUI de la aplicación del trabajo de la asignatura

En base al ejemplo visto en la sección 2.3.2 y los pasos que allí se describen para incorporar pruebas con Cucumber en un proyecto de Android Studio, vais a automatizar pruebas de aceptación sobre la aplicación del trabajo de la asignatura. Recordad que se deben preparar pruebas funcionales utilizando la técnica de *scenario testing*, y pruebas funcionales diseñadas con técnicas de caja negra (por ejemplo, particiones de equivalencia o pruebas de pares) sobre la validez de las reservas y el mantenimiento del precio de la reserva aunque se modifique posteriormente el precio del *quad* reservado.

Como punto de partida se facilita en el fichero «cucumber-notepad.zip» un ejemplo inicial de un test sobre la aplicación de gestión de notas (proporcionada como ejemplo en la práctica 2 de la asignatura de Ingeniería del Software). Este fichero contiene una definición de una *feature* para la creación de notas («notepad.feature»), la clase `RunCukesTest`

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

y la clase `RunStepsDefinition`. Este test se ha probado que funciona correctamente con un Pixel 3a API 35 (`compileSdk=35, targetSdk=34`).

Antes de empezar, se recomienda guardar una copia del original o trabajar sobre una rama si se utiliza un sistema de control de versiones.

2.4. Trabajo con la herramienta Exerciser Monkey

Android proporciona una herramienta denominada Exerciser Monkey, que envía eventos pseudoaleatorios a la interfaz de la aplicación que se desea probar. Es posible que, de esta forma, se detecten fallos y errores que de otra forma hubieran pasado inadvertidos, aunque al no seguirse ninguna técnica concreta al realizar la prueba, luego es difícil localizar dónde están los posibles defectos detectados.

Para ejecutar Exerciser Monkey, hay que ejecutar, desde un terminal o símbolo del sistema, la siguiente orden:

```
adb shell monkey -p paqueteDeLaApp -v numeroDeEventos
```

Donde *paqueteDeLaApp* es el nombre del paquete correspondiente a la aplicación a la que se le quieren mandar eventos, tal y como aparezca en la propiedad *applicationId* del fichero «`build.gradle`» de la aplicación, *-v* sirve para aumentar el nivel de verbosidad (una aparición implica un nivel 1 y se informa de los eventos enviados), y *numeroDeEventos* es el número de eventos que se le quieren mandar. Otro parámetro interesante es *-s semilla*, que permite especificar la semilla para inicializar el generador de números pseudo-aleatorios. Esto garantiza la reproducibilidad de los experimentos, ya que siempre se genera la misma secuencia de números aleatorios y por tanto, la misma secuencia de eventos mandados a la aplicación.⁹

La herramienta *adb* está dentro del directorio donde esté instalado el SDK de Android, en el subdirectorio «`Sdk/platform-tools/`». Es indiferente el directorio desde el que se ejecute la aplicación *adb*, pero el emulador tiene que estar previamente lanzado.

Utilizad Exerciser Monkey para mandar hacer pruebas de estrés aleatorias con la aplicación del trabajo. Exerciser Monkey finaliza cuando detecta algún error en la aplicación, informando sobre la semilla utilizada. Si se quiere evitar que Exerciser Monkey realice interacción con el sistema (es decir, sólo estamos interesados en la interacción y respuesta de nuestra aplicación), podemos utilizar la opción de Screen Pinning de Android [2].

⁹Ver descripción exhaustiva de todas las opciones en [1].

2.5. Trabajo con la herramienta Selenium

2.5.1. Trabajo con el entorno Selenium IDE

Selenium IDE es un complemento (plug-in) de navegadores como Firefox o Chrome que permite grabar interacciones con un navegador generando *scripts*, que posteriormente se pueden reproducir. El complemento se puede instalar a través de la página de descargas de Selenium.¹⁰

Una vez que se haya instalado el complemento, podréis acceder a la herramienta a través del icono *Se* que aparece en la barra de herramientas. Desde la página web que se abre podréis capturar y reproducir los *scripts* generados con esta herramienta. En el repositorio <https://github.com/unizar-vv-organization/unizar-vv-selenium> se facilita un ejemplo de estos *scripts* en el fichero «test_zaragoza.side» para acceder a la web del Ayuntamiento de Zaragoza.

Probad a utilizar la herramienta Selenium IDE para automatizar la descarga de algunos registros de metadatos disponibles a través del catálogo del Instituto Geográfico Nacional (IGN), el cual es accesible a través de la siguiente dirección: <http://www.ign.es/csw-inspire/srv/spa/main.home>.

Tal como se muestra en la figura 3, para poder descargar los metadatos se debe introducir una palabra clave en la caja de texto encabezada por *¿Qué busca?* y pulsar el botón *Buscar*. Posteriormente, cuando se muestren los resultados, se debe ir haciendo clic sobre el icono *XML* para ir descargando estos ficheros.

Se recuerda que para crear una primera versión del *script* con la herramienta Selenium IDE se puede utilizar la opción de grabación. Posteriormente, se pueden modificar algunas de las órdenes previamente grabadas para configurarlas correctamente.

2.5.2. Creación de *scripts* con Python y Selenium WebDriver

Selenium WebDriver es un driver con distintas implementaciones para interactuar con diferentes navegadores. Sobre la base de este WebDriver se pueden definir *scripts* en distintos lenguajes de programación (Java, C#, Ruby, Python) para interactuar con el navegador.

Siguiendo la documentación del paquete *selenium* para Python [5], en este apartado se propone la creación de un par de *scripts* en Python para interactuar con un navegador. Estos *scripts* están disponibles en el repositorio <https://github.com/unizar-vv-organization/unizar-vv-selenium> y se realizarán los siguientes pasos:

1. Instalad el driver para un navegador y sistema operativo específico. Por ejemplo,

¹⁰<http://docs.seleniumhq.org/download/>

Práctica 6: Automatización de pruebas sobre la interfaz de usuario



Figura 3: Catálogo de metadatos del Instituto Geográfico Nacional

en el caso del driver de Firefox para Linux, se puede realizar la instalación con el siguiente comando:

```
sudo apt install firefox-geckodriver
```

En la web del paquete *selenium* de Python se pueden encontrar los enlaces para la descarga de drivers de otros navegadores. En Windows, no hay que instalar Selenium Web Driver para acceder ni a Firefox, ni a Chrome ni a Edge.

2. Cread un proyecto con un entorno de desarrollo para Python, por ejemplo Visual Studio,¹¹ y/o configurar el entorno para incluir el paquete *selenium*:

```
pip install selenium
```

3. Cread un fichero Python (por ejemplo, «example.py») con un primer *script* impor-

¹¹Ver el tutorial de la herramienta en <https://code.visualstudio.com/docs/python/python-tutorial>

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

tando la librería *selenium* para acceder al portal del Ayuntamiento de Zaragoza:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
browser = webdriver.Firefox()
browser.get('https://zaragoza.es')
assert 'Zaragoza' in browser.title
# Desplegar la caja de búsqueda
searchButton = browser.find_element(By.LINK_TEXT, 'Buscador')
searchButton.click()
# Localizar la caja de búsqueda
elem = browser.find_element(By.NAME, 'query')
elem.send_keys('turismo' + Keys.RETURN)
response = input('Esperando para ver la página web, pulsa cualquier teclar: ')
browser.quit()
```

4. Cread un segundo fichero Python (por ejemplo, «GoogleTestCase.py») con un *script* utilizando la librería *unittest* de Python, la cual facilita un entorno de automatización similar al de JUnit para Java:

```
import unittest
from selenium import webdriver

class GoogleTestCase(unittest.TestCase):
    def setUp(self):
        self.browser = webdriver.Firefox()
        self.addCleanup(self.browser.quit)
    def test_page_title(self):
        self.browser.get('https://www.zaragoza.es')
        self.assertIn('La Web de la Ciudad de Zaragoza', self.browser.title)
        response = input('Esperando para ver la página web, pulsa cualquier tecla: ')
if __name__ == '__main__':
    unittest.main(verbosity=2)
```

5. Podéis continuar mejorando los *scripts* revisando tutoriales más avanzados sobre el uso del paquete *selenium* [4].

2.5.3. Creación de *scripts* con Behave, Python y Selenium WebDriver

También es posible escribir casos de prueba con el lenguaje Gherkin (el utilizado por la herramienta Cucumber) y generar posteriormente *scripts* con Python y Selenium WebDriver para interactuar con aplicaciones web a través de un navegador. La herramienta Behave [3] permite interpretar los casos de prueba con Gherkin e invocar el *script* en Python.

Dentro de la carpeta «behave» del repositorio <https://github.com/>

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

unizar-vv-organization/unizar-vv-selenium se proporciona un pequeño ejemplo para interactuar con la página web del Ayuntamiento de Zaragoza. Asumiendo que ya se ha configurado un entorno virtual para ejecutar los *scripts* de Python anteriores, se realizarán los siguientes pasos adicionales:

1. Instalar el paquete *behave* dentro del entorno virtual de Python:

```
pip install behave
```

2. Definir un fichero *.feature* como por ejemplo «*buscar.feature*» con el caso de pruebas:

Feature: Buscar en la web del Ayuntamiento de Zaragoza

Scenario: Un usuario busca la palabra turismo en la web del Ayuntamiento de

↪ Zaragoza

Given El usuario está en la página inicial de la web del Ayuntamiento de

↪ Zaragoza

When Hace clic en el botón de búsqueda

When Introduce como palabra de búsqueda "turismo"

Then Aparece la página de resultados

3. Implementar los pasos en una subcarpeta denominada «*steps*» dentro de la carpeta donde se ha definido el fichero *.feature*. Se van implementando los pasos en el fichero «*steps.py*»:

```
from behave import *
from functions import MyFunctions
@given(u'El usuario está en la página inicial de la web del Ayuntamiento de
↪ Zaragoza')
def step_impl(context):
    MyFunctions.enterMainPage(context)
@when(u'Hace clic en el botón de búsqueda')
def step_impl(context):
    MyFunctions.clickSearchButton(context)
@when(u'Introduce como palabra de búsqueda "{keyword}")
def step_impl(context, keyword):
    MyFunctions.enterKeyword(context, keyword)
@then(u'Aparece la página de resultados')
def step_impl(context):
    MyFunctions.resultsPageIsOpen(context)
```

4. Desde el fichero «*steps.py*» se hace referencia al fichero «*functions.py*»:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
link = "https://www.zaragoza.es"
```

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

```
class MyFunctions():
    @classmethod
    def setup_class(self):
        print("\nstart browser for test suite..")
        self.driver = webdriver.Firefox()
    @classmethod
    def teardown_class(self):
        print("quit browser for test suite..")
        self.driver.quit()
    def enterMainPage(self):
        self.driver = webdriver.Firefox()
        self.driver.get(link)
    def clickSearchButton(self):
        searchButton = self.driver.find_element(By.LINK_TEXT, 'Buscador')
        searchButton.click()
    def enterKeyword(self, keyword):
        elem = self.driver.find_element(By.NAME, 'query') # Find the search box
        elem.send_keys(keyword + Keys.RETURN)
    def resultsPageIsOpen(self):
        element = WebDriverWait(self.driver, 10).until(EC.title_contains("
        ↪ Buscador. Ayuntamiento de Zaragoza"))
```

«functions.py» interacciona con la aplicación web a través del driver del navegador (SeleniumWebDriver) utilizando el paquete *selenium*.

- La ejecución del caso de pruebas consiste en la ejecución del fichero *.feature*. Desde la terminal del entorno de desarrollo se debe lanzar el programa *behave*:

```
behave buscar.feature
```

3. Entrega de la práctica

No se solicitará la entrega explícita de *software* o documentación para esta sesión de prácticas, pero en la entrega final del trabajo de la asignatura sí que se deberá incluir como mínimo:

- La automatización con Espresso de la prueba funcional de sistema sobre la navegación de la aplicación, diseñada mediante la técnica de caminos.
- La automatización con Cucumber de las pruebas de aceptación funcionales que incluyen casos de prueba representativos de los requisitos de usuario utilizando la técnica *scenario testing* mencionada en el tema de “Evaluación y testing de Interfaces Persona Ordenador”).
- La automatización con Cucumber de las pruebas de aceptación funcionales sobre la validez de las reservas y el mantenimiento del precio de la reserva (aunque se

Práctica 6: Automatización de pruebas sobre la interfaz de usuario

modifique posteriormente el precio del *quad* reservado) diseñadas con técnicas de caja negra como particiones de equivalencia o pruebas de pares.

Referencias

- [1] Android Developers. UI/Application Exerciser Monkey. <https://developer.android.com/studio/test/monkey>, 2023.
- [2] Google. Android Help - Pin & unpin screens. <https://support.google.com/android/answer/9455138?hl=en>, 2025.
- [3] Benno Rice Jens Engel and Richard Jones. Welcome to behave! <https://behave.readthedocs.io/en/latest/>, 2024.
- [4] Baiju Muthukadan. Selenium with Python. <https://selenium-python.readthedocs.io/index.html>, 2024.
- [5] Python Software Foundation. selenium 4.31.0 - Official Python bindings for Selenium WebDriver. <https://pypi.org/project/selenium/>, 2025.
- [6] T. Sundberg. Cucumber-JVM – not just for testing GUIs. <http://www.thinkcode.se/blog/2012/11/01/cucumberjvm-not-just-for-testing-guis>, 2012.
- [7] Lars Vogel. Android cross component with the UI Automator framework - Tutorial. <https://www.vogella.com/tutorials/AndroidTestingUIAutomator/article.html>, 2016.
- [8] Lars Vogel. Android user interface testing with Espresso - Tutorial. <https://www.vogella.com/tutorials/AndroidTestingEspresso/article.html>, 2016.